Автотесты LMS: инструкция для QA

Как устроен проект, как запускать и как добавлять тесты. Актуально для ветки `office/autotests`, сентябрь 2026. Подробности по окружениям, Docker и БУК есть в README.md, здесь про то, как писать.

1. Как это работает в двух словах

Тесты написаны на pytest. API-тесты ходят на портал через `requests`, UI-тесты управляют браузером через Playwright. Отчёты в Allure.

Один запуск, один стенд: ключ `--env develop|stage|prod` выбирает адреса, учётки и токены. Перед тестами фреймворк логинится по логину и паролю из `.env`, сессия кэшируется в папке `.cache/`, поэтому каждый прогон не логинится заново. Для UI та же сессия превращается в storage state браузера, и страницы открываются уже авторизованными.

Каждый тест независим: данные для него создаёт фикстура через API, а после теста та же фикстура их удаляет. Названия сущностей генерируются уникальными, так что тесты можно гонять параллельно.

2. Дерево проекта

```
auto_tests_vshgu_cloude/
  conftest.py            общие фикстуры: окружение, сессии, клиенты, created_* с
очисткой
  pytest.ini             маркеры, testpaths, allure
  requirements.txt
  Dockerfile, docker-compose.yml, scripts/run.sh, scripts/run.bat
  .env                   учётки и токены, в git не попадает (.env.example — список
переменных)

  config/
    environments.py      реестр окружений, логин, кэш сессий, storage_state для UI
    settings.py          старые алиасы для develop/stage, не использовать в новых
тестах

  api/
    client.py            ApiClient: сессия, базовый URL, заголовки, логирование
запросов
    endpoints/           по модулю на сущность: programs.py, streams.py, users.py,
cas, buk, lpd…
    helpers/
      expectations.py    expect_status(resp, 200): единая проверка кода со всем
телом ответа в ошибке
      data_generator.py  generate_*_payload(): уникальные тела запросов
      constants.py       справочники: локации и организации CAS и т. п.
      user_prepare_api.py подготовка пользователя: согласие ПД, закрытие онбординга

  ui/
    pages/               Page Object: base_page.py, login_page.py, admin_*_page.py,
```

```
user_*_page.py…

  tests/
    api/                 API-тесты по областям: programs, streams, users, cas, buk,
lpd,
                         create_activity (courses, homeworks, lpd…), polls,
certificates, tests…
    ui/                  UI-тесты по тем же областям + practical_tasks,
knowledge_base, onboarding
    e2e/                 сквозные сценарии с маркером env("stage")

  test_data/             файлы для загрузок: обложки, docx, xlsx, архивы SCORM и
tincan
  test_data_generation/  скрипты подготовки данных на стенде (CAS и др.)
  utils/                 logger.py, retry.py
  results/               результаты прогонов через scripts/run.sh, в git не
попадают
```

3. Слои и правила

Слой	Где	Правило
Эндпоинты	`api/endpoints/*.py`	Один класс на сущность, метод делает запрос и проверяет код через `expect_status`. Никакой бизнес-логики теста внутри
Генераторы данных	`api/helpers/data_generator.py`	Каждое тело запроса с уникальным именем, через timestamp
Фикстуры	`conftest.py`	`created_program`, `created_stream`, `created_user` и т. д. Возвращают функцию-фабрику, всё созданное удаляют после теста
Page Object	`ui/pages/*.py`	Локаторы и действия страницы. Ожидания через `expect(...)`, без `sleep` и `wait_for_timeout`
Тесты	`tests/`	Только сценарий: шаги Allure, вызовы фикстур и страниц, проверки

Что не делать: не хардкодить учётки и адреса, не читать `settings.py` в новых тестах, не ждать `networkidle` без необходимости, не оставлять созданные данные на стенде.

4. Запуск

Локально, из корня проекта:

```
.venv/bin/python -m pytest --env=develop -m smoke
.venv/bin/python -m pytest --env=stage tests/api/buk
.venv/bin/python -m pytest --env=develop
```

```
tests/ui/users/test_user_login_navigation.py
.venv/bin/python -m pytest --env=develop -n auto
```

UI-тесты локально открывают окно браузера. Переменная `HEADLESS=1` включает безоконный режим, в Docker она стоит по умолчанию.

Маркеры: `api`, `ui`, `smoke`, `practical_tasks`, `groups`, `onboarding`, `knowledge_base`, `reviewer`, `env("stage")`. Тест с `env("stage")` пропускается при запуске на другом стенде. Prod без `-m smoke` не запустится, это защита от полного прогона на бою.

Через Docker и с отчётом Allure:

```
docker compose run --rm tests --env develop -m smoke
scripts/run.sh --env stage -m "smoke and api"
```

Результаты `run.sh` лежат в `results/<env>/<timestamp>/`, отчёт в `results/<env>/allure-report/`.

5. Как добавить API-тест

Пример: тест на новую сущность «рассылки».

Шаг 1. Эндпоинт в `api/endpoints/newsletters.py`:

```
from api.client import ApiClient
from api.helpers.expectations import expect_status


class NewslettersEndpoint:
    def __init__(self, client: ApiClient):
        self.client = client

    def create(self, payload: dict) -> dict:
        resp = self.client.post("/newsletters", json=payload)
        expect_status(resp, (200, 201))
        return resp.json()["data"]

    def delete(self, newsletter_id: int) -> bool:
        resp = self.client.delete(f"/newsletters/{newsletter_id}")
        return resp.status_code in (200, 204)
```

Шаг 2. Генератор в `api/helpers/data_generator.py`:

```
def generate_newsletter_payload(name: str = None, **kwargs) -> dict:
    ts = int(time.time() * 1000)
    return {"name": name or f"Автотест рассылка {ts}", **kwargs}
```

Шаг 3. Фикстуры в `conftest.py`, по образцу `created_program`:

```
@pytest.fixture
def newsletters_endpoint(api_client):
    return NewslettersEndpoint(api_client)
```

```
@pytest.fixture
def created_newsletter(newsletters_endpoint):
    ids = []

    def _create(**kwargs):
        data = newsletters_endpoint.create(generate_newsletter_payload(**kwargs))
        ids.append(data["id"])
        return data

    yield _create
    for newsletter_id in ids:
        newsletters_endpoint.delete(newsletter_id)
```

Шаг 4. Тест в `tests/api/newsletters/test_create_newsletter_api.py`:

```
import allure
import pytest
from api.helpers.expectations import expect_status


@allure.feature("Newsletters API")
class TestCreateNewsletterAPI:

    @allure.title("Создание рассылки с валидными данными")
    @pytest.mark.api
    @pytest.mark.smoke
    def test_create_newsletter_success(self, created_newsletter):
        with allure.step("1. Создать рассылку"):
            newsletter = created_newsletter()
        with allure.step("2. Проверить ответ"):
            assert newsletter["id"]
            assert newsletter["attributes"]["name"].startswith("Автотест рассылка")

    @allure.title("Обычный пользователь не может создать рассылку")
    @pytest.mark.api
    def test_create_forbidden_for_regular_user(self, env, created_user,
newsletters_endpoint):
        user = created_user()
        session = login(env, user["email"], user["password"])
        client = ApiClient(session=session, base_url=env.api_base_url,
cookie_name=env.cookie_name)
        resp = client.post("/newsletters", json=generate_newsletter_payload())
        expect_status(resp, 403)
```

Негативные проверки делаются так же: вызываем клиент напрямую и проверяем код через `expect_status`, чтобы в ошибке было тело ответа.

6. Как добавить UI-тест

Шаг 1. Page Object в `ui/pages/newsletters_page.py`. Наследуемся от `BasePage`, локаторы в атрибутах, действия в методах, ожидания через `expect`:

```
from playwright.sync_api import expect
from ui.pages.base_page import BasePage


class AdminNewslettersPage(BasePage):
    URL = "/admin/newsletters"

    def open_list(self, base_url: str):
        self.open(f"{base_url}{self.URL}")
        expect(self.page.get_by_role("heading", name="Рассылки")).to_be_visible()

    def search(self, text: str):
        self.page.get_by_test_id("km-search").fill(text)
        self.page.keyboard.press("Enter")

    def row(self, name: str):
        return self.page.locator("tr.vs-table__row", has_text=name)
```

Шаг 2. Тест в `tests/ui/newsletters/test_admin_newsletters_ui.py`. Данные создаём через API, браузер берём уже авторизованным:

```
import allure
import pytest
from playwright.sync_api import expect
from ui.pages.newsletters_page import AdminNewslettersPage


@allure.feature("UI: рассылки")
class TestAdminNewslettersUI:

    @allure.title("Созданная через API рассылка видна в списке админки")
    @pytest.mark.ui
    def test_newsletter_visible_in_admin_list(self, admin_page, env,
created_newsletter):
        newsletter = created_newsletter()
        page = AdminNewslettersPage(admin_page)
        with allure.step("1. Открыть список рассылок"):
            page.open_list(env.ui_base_url)
        with allure.step("2. Найти рассылку по названию"):
            page.search(newsletter["attributes"]["name"])
            expect(page.row(newsletter["attributes"]["name"])).to_be_visible()
```

Фикстуры браузера: `admin_page` авторизована под администратором, `user_page` под слушателем, `page` чистая, для тестов формы входа. При падении UI-теста скриншот прикладывается в Allure сам.

Если нужен новый маркер, добавьте его в `pytest.ini`, иначе pytest откажется запускать тест: маркеры строгие.

7. Как отлаживать

- Один тест: `.venv/bin/python -m pytest --env=develop tests/api/programs/test_create_program_api.py -k success -q`.
- UI с окном браузера: не задавать `HEADLESS`, при необходимости `PWDEBUG=1` для пошагового режима Playwright.
- Тело ответа при неверном коде уже в тексте ошибки, смотреть в отчёте или в консоли.
- Скриншоты UI-падений в `allure-results/`, отчёт: `allure serve allure-results`.
- Если все тесты вдруг падают с 401, протухла сессия: удалить файлы окружения в `.cache/`, прогон залогинится заново. Если не помогает, проверить логин и пароль в `.env`.

8. Чек-лист перед коммитом

1. Тест проходит на develop минимум два раза подряд.
2. Данные после теста удалены, на стенде не остаётся мусора с именем «Автотест…».
3. Нет хардкода учёток, адресов, паролей.
4. Маркеры стоят: `api` или `ui`, при необходимости `smoke` и область.
5. Заголовок и шаги Allure на русском, читаются без кода.
6. Для UI: ожидания через `expect`, без `sleep`.